Department of Computer Engineering
University of Peradeniya
CO2030 - Data Structures and Algorithms II

Lab 01: Implementing Trees and Binary Search Trees

Learning Outcomes

By the end of this laboratory session, you should be able to:

1. Understand and implement the Tree Abstract Data Type (ADT)
2. Implement various tree traversal algorithms (pre-order, in-order, post-order)
3. Compare and implement both recursive and iterative traversal methods
4. Implement Binary Search Tree (BST) operations (insertion, deletion, search)
5. Analyze time complexity of BST operations
6. Apply tree structures to solve real-world problems

Prerequisites

Before beginning this laboratory session, ensure you have:

- Familiarity with Python programming
- Understanding of recursion and recursive algorithms
- Knowledge of linked data structures
- Python 3.8 or higher installed on your system

1. Understanding Tree Terminology and Basic Implementation

1.1 Tree Terminology Review

Before implementation, ensure you understand these key concepts:

Root: The topmost node in a tree
Parent: A node that has children
Child: A node that has a parent
Leaf: A node with no children
Sibling: Nodes with the same parent
Degree: Number of children of a node
Height: Length of the longest path from a node to a leaf
Depth: Length of the path from the root to a node
Level: All nodes at the same depth

1.2 Task: Implement a Basic Tree Node

Create a `TreeNode` class that can be used to build tree structures.

1.2.1 Requirements

- **Class:** `TreeNode`
- **Methods to Implement:**
  - `add_child(child_node)` - Add a child to this node
  - `remove_child(child_node)` - Remove a child from this node
- **Pre-implemented Methods (provided):**
  - `__init__(data)`
  - `get_children(): list`
  - `get_data()`
  - `is_leaf(): bool`
  - `get_degree(): int`
  - `print_tree(level=0)`

1.2.2 Template Structure (Python)

```python
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None

    def add_child(self, child_node):
        # TODO: Implement
        pass

    def remove_child(self, child_node):
        # TODO: Implement
        pass

    # Rest of the code
```

Exercise 1.1

Complete the `add_child` and `remove_child` methods in the TreeNode class.

Exercise 1.2

Create a simple tree structure representing a file system:

```
Root (/)
├── home
│   ├── user1
│   └── user2
├── var
│   ├── log
│   └── tmp
└── etc
```

Deliverable

Screenshot of your tree structure and code implementation.

2. Tree Traversal Algorithms

2.1 Understanding Traversal Methods

Tree traversal is the process of visiting each node in a tree data structure exactly once. The three main depth-first traversal methods are:

1. **Pre-order (Root-Left-Right):** Visit root, traverse left subtree, traverse right subtree
2. **In-order (Left-Root-Right):** Traverse left subtree, visit root, traverse right subtree
3. **Post-order (Left-Right-Root):** Traverse left subtree, traverse right subtree, visit root

2.2 Task: Implement Recursive Traversals

Add the following methods to a `Tree` class:

```
class Tree:
    def __init__(self, root):
        self.root = root

    def pre_order_recursive(self, node=None):
        """Pre-order traversal: Root -> Children"""
        # TODO: Implement pre-order traversal
        pass

    def post_order_recursive(self, node=None):
        """Post-order traversal: Children -> Root"""
        # TODO: Implement post-order traversal
        pass
```

Exercise 2.1

Implement recursive pre-order and post-order traversals in the Tree class.

Exercise 2.2

Compare the performance and readability of recursive vs. iterative (already given in the template) approaches. Create a table:

Aspect	Recursive	Iterative
Code Readability		
Memory Usage		
Performance		
Ease of Implementation		

Deliverable

Completed recursive traversal implementations with test outputs and comparison table.

3. Binary Trees - Foundation

3.1 Binary Tree Node

A binary tree is a tree where each node has at most two children (left and right).

```
class BinaryTreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

3.2 Binary Tree Operations

Create a `BinaryTree` class with traversal operations:

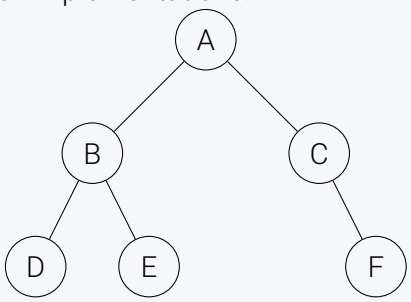
```
class BinaryTree:
    def __init__(self, root=None):
        self.root = root

    def in_order(self, node=None):
        """In-order traversal: Left -> Root -> Right"""
        # TODO: Implement in-order traversal
        pass

    def pre_order(self, node=None):
        """Pre-order traversal: Root -> Left -> Right"""
        # TODO: Implement pre-order traversal
        pass

    def post_order(self, node=None):
        """Post-order traversal: Left -> Right -> Root"""
        # TODO: Implement post-order traversal
        pass

    # Rest of the code
```

Exercise 3.2

Implement all three traversal methods (in-order, pre-order, post-order) for binary trees.

Exercise 3.3

Create a binary tree and verify your implementations:



Expected outputs:

- Pre-order: A B D E C F
- In-order: D B E A C F
- Post-order: D E B F C A
- Height: 3 (using provided method)
- Total nodes: 6 (using provided method)
- Leaf nodes: 3 (using provided method)

Deliverable

Complete implementation with test results showing correct traversal outputs and tree metrics.

4. Binary Search Trees - Implementation

4.1 Understanding BST Properties

A Binary Search Tree is a binary tree with the following properties:

- All nodes in the left subtree have values **less than** the root
- All nodes in the right subtree have values **greater than** the root
- Both left and right subtrees are also BSTs
- No duplicate values

4.2 Task: Implement BST Insert Operation

```
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        """Public insert method"""
        self.root = self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Recursive helper to insert data"""
        if node is None:
            return BinaryTreeNode(data)

        # TODO: Implement recursive insert
        # Compare data with node.data
        # If data < node.data: recursively insert in left subtree
        # If data > node.data: recursively insert in right subtree
        # Return node
        pass

    # Rest of the code
```

Exercise 4.1

Complete the recursive `_insert_recursive` method.

Exercise 4.2

Insert the following values into your BST: 50, 30, 70, 20, 40, 60, 80. Draw the resulting tree structure.

4.3 Task: Implement BST Search Operation

```
def search(self, data):
    return self._search_recursive(self.root, data)

def _search_recursive(self, node, data):
    """Recursive helper to search for data"""
    # TODO: Implement recursive search
    pass

def search_iterative(self, data):
    """Iterative search implementation"""
    # TODO: Implement iterative search using a loop
    pass
```

Exercise 4.3

Implement both recursive and iterative search methods.

Exercise 4.4

Test your search on the BST from Exercise 4.2. Search for:

- Values that exist: 30, 60, 80
- Values that don't exist: 25, 45, 75

Verify both recursive and iterative versions produce the same results.

4.4 Task: Implement BST Deletion Operation

Deletion is the most complex BST operation. There are three cases:

1. **Node is a leaf:** Simply remove it
2. **Node has one child:** Replace node with its child
3. **Node has two children:** Find in-order successor (smallest in right subtree), replace node's value, and delete the successor

```
def delete(self, data):
    self.root = self._delete_recursive(self.root, data)

def _delete_recursive(self, node, data):
    """Recursive helper to delete data"""
    if node is None:
        return None

    # Find the node to delete
    if data < node.data:
        node.left = self._delete_recursive(node.left, data)
    elif data > node.data:
        node.right = self._delete_recursive(node.right, data)
    else:
        # TODO: Node found! Handle three cases
        pass

    return node
```

Exercise 4.5

Implement the `delete` method with all three cases

Exercise 4.6

Test deletion on your BST:

1. Delete a leaf node (20)
2. Delete a node with one child (create a suitable test case)
3. Delete a node with two children (50)

After each deletion, perform in-order traversal to verify BST property is maintained.

5. Building a Student Database

5.1 Problem Statement

Design and implement a student management system using BST where students are ordered by their ID numbers.

5.2 Requirements

```python
class Student:
    def __init__(self, student_id: int, name: str, gpa: float):
        self.student_id = student_id
        self.name = name
        self.gpa = gpa

    def __lt__(self, other):
        # TODO: Implement less than comparison
        pass

    def __gt__(self, other):
        # TODO: Implement greater than comparison
        pass

    def __eq__(self, other):
        # TODO: Implement equality comparison
        pass


class StudentDatabase:
    def __init__(self):
        self.database = BinarySearchTree()

    def add_student(self, student: Student):
        # TODO: Insert student into BST
        pass

    def find_student(self, student_id: int) -> Student:
        # TODO: Search for student by ID
        pass

    def remove_student(self, student_id: int):
        # TODO: Delete student from BST
        pass

    def display_all_students(self):
        # TODO: In-order traversal to display sorted list
        pass

    def display_top_performers(self, min_gpa: float):
        # TODO: Find and display students with GPA >= min_gpa
        pass
```

Exercise 5.1

Implement the `Student` class with proper comparison methods.

Exercise 5.2

Implement the `StudentDatabase` class with all required methods.

Exercise 5.3

Create a test program that:

1. Adds at least 10 students with varying IDs and GPAs
2. Searches for specific students
3. Displays all students in sorted order (by ID)
4. Removes a student and verifies deletion
5. Displays top performers (GPA $\geq$ 3.7)

5.2.1 Sample Test Data

```
Student(12345, "Alice Johnson", 3.8)
Student(23456, "Bob Smith", 3.2)
Student(34567, "Charlie Brown", 3.9)
Student(11111, "Diana Prince", 4.0)
Student(99999, "Eve Wilson", 3.5)
# Add 5 more students
```

Deliverable

Complete student database system with test results.

6. String-Based Student IDs and Alternative Data Structures

6.1 Problem: Changing ID Format

In Part 5, we used simple numeric IDs (12345, 23456, etc.) for students. However, in the real University of Peradeniya system, student IDs follow a structured string format:

E/YY/XXX (e.g., E/19/008, E/20/145, E/21/032)

Where:

- **E** = Faculty (Engineering)
- **YY** = Year of enrollment (19, 20, 21, etc.)
- **XXX** = Student number within that year

Critical Reflection Questions:

1. Can we still use a BST with string IDs like "E/19/008"?
2. What operations become inefficient with BST when IDs are strings?
3. What if we need to find "all students from year 19" (prefix "E/19/")?

6.2 BST Limitations with String IDs

Using BST with string IDs has several drawbacks:

Operation	BST Performance with Strings
Search exact ID ("E/19/008")	O(m * log n) where m = string length Must compare entire strings at each node
Find all students from year 19 ("E/19/...")	O(n * m) - Must scan entire tree No efficient prefix-based filtering
Autocomplete ID entry	Very inefficient - no prefix support
Range query ("E/19/001" to "E/19/999")	Possible but string comparisons are slow

6.3 Alternative Data Structure

Exercise 6.3: Research and Propose a Better Data Structure

Given the limitations of BST for string-based IDs with prefix operations:

1. **Research:** What tree-based data structure is specifically designed for efficient string operations and prefix searches?
2. **Explain:** How does this data structure store strings differently than BST?
3. **Analyze:** Why is it better for operations like:
   - Finding all students from year 19 ("E/19/...")
   - Autocomplete functionality
   - Prefix validation
4. **Create a comparison table:** Compare your proposed data structure with BST using the format below:

Operation	BST (String IDs)	Your Data Structure
Search exact ID	O(m * log n)	?
Insert new student	O(m * log n)	?
Find all from year 19 ("E/19/...")	O(n * m) scan all	?
Autocomplete ("E/2...")	Not practical	?
Sorted traversal	Efficient	?
Space complexity	O(n * m)	?

Hint: *Think about data structures that store strings character-by-character and share common prefixes.*

Submission Guidelines

Report Requirements

Your lab report should be a concise document that demonstrates your understanding and implementation of tree and BST concepts. The report should include:

Report Structure

1. **For Each Exercise :**
   - **Exercise Number and Title** (e.g., Exercise 1.1: TreeNode Implementation)
   - **Implementation File Reference** (e.g., "Implemented in `tree_node.py`")
   - **Approach/Algorithm**: Brief explanation of your implementation approach (2-3 sentences)
   - **Results**: Screenshots or output showing successful execution (if possible)
   - **Analysis**: Answer any questions posed in the exercise
2. **Comparison Tables and Diagrams**
   - Include all requested comparison tables (e.g., Exercise 2.3.3)
   - Draw tree structures where requested (e.g., Exercise 4.3)
   - Use clear, labeled diagrams

What NOT to Include

Don't include:

- Copy-pasted entire source code files in the report
- Redundant screenshots of the same output

What TO Include

Do include:

- References to your code files by name (e.g., `binary_search_tree.py`)
- Small code snippets / pseudo codes only when explaining a specific algorithm or approach (if necessary)

Submission Format

Create a ZIP file named `CO2030_Lab01_E/XX/YYY.zip` containing:

```
Codes/
├── tree_node.py
├── tree.py
├── binary_tree_node.py
├── binary_tree.py
├── binary_search_tree.py
├── student.py
├── student_database.py
├── main.py (with all tests)
├── tests/
│   └── test_all.py
└──screenshots/

CO2030_Lab01_E/XX/YYY_report.pdf
```

Evaluation

In-Class Evaluation

Important: Your work will be evaluated in two parts:

- **In-Class Evaluation (Sections 1-4):**
  - Section 1: Understanding Tree Terminology and Basic Implementation
  - Section 2: Tree Traversal Algorithms
  - Section 3: Binary Trees - Foundation
  - Section 4: Binary Search Trees - Implementation
  - Demonstrate your working implementations to the instructors during the lab session
  - Be prepared to explain your code and design decisions
  - Answer questions about algorithms and complexity
  - Show test results and functionality
- **Homework Submission (Sections 5,6 + Report):**
  - Section 5: Building a Student Database
  - Section 6: Alternative Data Structure
  - Complete implementations with comprehensive testing
  - Submit code files and lab report as described above

Deadline

Important

In-Class Evaluation: During your scheduled lab session
Submission: [Date to be announced]

Appendix A: Quick Reference

6.4 Tree Traversal Pseudocode

6.4.1 Pre-order (Root-Left-Right)

```
preorder(node):
    if node is null:
        return
    visit(node)
    preorder(node.left)
    preorder(node.right)
```

6.4.2 In-order (Left-Root-Right)

```
inorder(node):
    if node is null:
        return
    inorder(node.left)
    visit(node)
    inorder(node.right)
```

6.4.3 Post-order (Left-Right-Root)

```
postorder(node):
    if node is null:
        return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

6.5 BST Operations Pseudocode

6.5.1 Search

```
search(node, value):
    if node is null:
        return false
    if node.data == value:
        return true
    if value < node.data:
        return search(node.left, value)
    else:
        return search(node.right, value)
```

6.5.2 Insert

```
insert(node, value):
    if node is null:
        return new Node(value)
    if value < node.data:
        node.left = insert(node.left, value)
    else if value > node.data:
        node.right = insert(node.right, value)
    return node
```

Appendix B: Implementation Summary

What You Need to Implement

Section	Component	Methods to Implement
Section 1: Tree Node		
TreeNode	Class	`add_child()`, `remove_child()`
Section 2: Tree Traversals		
Tree	Traversals	`pre_order_recursive()`, `post_order_recursive()`
Section 3: Binary Trees		
BinaryTreeNode	Class	Full initialization
BinaryTree	Traversals	`in_order()`, `pre_order()`, `post_order()`
Section 4: BST		
BST	Insert	`_insert_recursive()`
BST	Search	`_search_recursive()`, `search_iterative()`
BST	Delete	`_delete_recursive()`, `_find_min()`
Section 5: Student Database		
Student	Class	All comparison methods
StudentDatabase	All methods	Complete implementation
Section 6: Alternative		
Research	Analysis	Data structure comparison

What Is Already Provided

- **TreeNode:** `__init__()`, `get_children()`, `get_data()`, `is_leaf()`, `get_degree()`, `print_tree()`
- **Tree:** `pre_order_iterative()`, `post_order_iterative()`
- **BinaryTree:** `height()`, `count_nodes()`, `count_leaves()`
- **BST:** `insert_iterative()`, `find_min()`, `find_max()`, `is_valid_bst()`, `in_order()`

End of Lab Sheet

Good luck with your implementation! Remember, understanding trees is fundamental to many advanced data structures and algorithms. Focus on the methods you need to implement, and use the provided implementations to verify your work.